

Aula 18 - Segment Tree

O problema que a SegTree resolve

A SegTree é uma das estruturas de dados mais úteis que podemos aprender. Imagine o seguinte array:

$$v = [5, 2, 7, 3, 6, 1, 4, 8]$$

E você precisa responder várias perguntas como:

- Soma de `l até r`
- Mínimo de `l até r`
- Máximo de `l até r`
- GCD de `l até r`
- Some 1 na posição 4 do vetor, depois responda qual a soma de `l até r`
- Subtraia 2 da posição 6 do vetor, depois responda qual a soma de `l até r`

Em que `l` e `r` são índices quaisquer dentro de `v` ($l \leq r$).

A solução ingênua cai em $O(n)$ para cada uma das Q consultas, entregando pra gente $O(nQ)$. Isso não é muito legal. Ao implementar uma Prefix Sum (que vimos nas aulas passadas), a gente consegue reduzir o custo de algumas consultas como "soma de `l até r`" para $O(1)$, mas as perguntas que envolvem atualizar algum valor do vetor ainda são $O(n)$, dado que teríamos que refazer o vetor de Prefix Sum.

Nesse cenário, surge a Segment Tree. Ela resolve problemas em que precisamos de:

- Consultas rápidas em intervalos
- Atualizações rápidas em posições (ou até em intervalos, em versões mais avançadas da SegTree)

Ao implementar uma SegTree, conseguimos os custos:

- $O(n)$ para construir inicialmente a árvore
- $O(\log n)$ para qualquer consulta ou atualização

OBS.: A SegTree que vou mostrar nessa aula funciona apenas para atualizações em uma única posição do vetor. Para atualizações de intervalos (ex.: some 3 em todos os elementos entre l e r) precisamos de outra técnica chamada de SegTree com Lazy Propagation.

Intuição

A ideia aqui é criar uma árvore binária (cada nó tem exatamente dois filhos, exceto as folhas, obviamente), em que cada nó representa um intervalo do vetor v original.

Vamos seguir uma regra estritamente: se um nó representa um intervalo $[l, r]$ do vetor original, seu filho esquerdo vai representar o intervalo $[l, m]$ do vetor original, enquanto seu filho direito vai representar o intervalo $[m + 1, r]$ do vetor original, sendo m o meio do intervalo $[l, r]$.

Logo, voltando para o exemplo em que temos o array $[5, 2, 7, 3, 6, 1, 4, 8]$, a raiz da nossa SegTree vai representar o intervalo $[0, 7]$ enquanto o filho esquerdo da raiz vai representar o intervalo $[0, 3]$ e o filho direito vai representar o intervalo $[4, 7]$. Daí, o filho esquerdo do filho esquerdo vai representar o intervalo $[0, 1]$ e o filho direito do filho esquerdo vai representar o intervalo $[2, 3]$, e assim por diante, até chegarmos nas folhas, que representarão intervalos do tipo $[n, n]$ (um intervalo de tamanho 1, que significa o próprio elemento do vetor).

O que cada nó guarda exatamente?

Eu disse ali em cima que cada nó “representa” um intervalo, mas qual valor esse nó guarda? A resposta é: depende do problema.

Podemos fazer uma SegTree em que cada nó guarda a soma do intervalo, ao passo que podemos fazer uma SegTree em que cada nó guarda o valor mínimo de um intervalo, e assim por diante.

Implementação

Ideias iniciais

Normalmente, implementar uma estrutura de dados na forma de árvore exige declarar nós com ponteiros, daí, teríamos que mexer com ponteiros apontando para filhos e para pais, é uma dor de cabeça.

No entanto, podemos “simular” uma estrutura de árvore usando simplesmente um vetor bizarramente grande.

Se temos um vetor original v de tamanho n , para simular a SegTree ligada a esse vetor, precisamos de um novo vetor, que chamaremos de $tree$, de tamanho $4n$, para garantir que não haverá overflow.

Assim, precisamos seguir estritamente uma regra para representar os filhos de um nó:

- Se um nó está na posição p do vetor original:
 - O filho esquerdo desse nó estará na posição $2p$
 - O filho direito desse nó estará na posição $2p + 1$

Notem que, a posição dos filhos é definida a partir da posição do pai, mas, a posição de todo mundo é definida a partir da posição dos elementos individuais no vetor original, e quem são os elementos individuais na SegTree? São as folhas.

O que eu quero dizer com isso é: para saber as posições dos pais no vetor $tree$, precisamos saber primeiro a posição das folhas, ou seja, para construir a nossa árvore, precisamos começar de baixo e subir na árvore.

Adicionalmente, quando formos setar os valores de cada nó na árvore, o valor que eu quero salvar depende do valor que está salvo nos filhos dele. Vamos tomar como exemplo a soma do intervalo $[l, r] \rightarrow$ sabemos que ele depende da soma do intervalo $[l, m]$ e da soma do intervalo $[m + 1, r]$.

Ou seja, todas as operações que queremos fazer tem o mesmo comportamento, precisamos percorrer a árvore de baixo para cima.

Isso determina um comportamento **recursivo**. Podemos começar uma recursão na raiz e adentrar na árvore o mais fundo possível até chegar nas folhas (casos-

base) e aí sim voltar.

Implementando uma SegTree

Dito isso, vamos implementar as três operações fundamentais de uma SegTree (`build`, para construir a árvore com os valores iniciais, `query`, para consultar uma informação em um intervalo, e `update`, para alterar um valor e já atualizar as informações de intervalos que contém aquele valor) usando recursão, que é mais simples do que a forma iterativa.

Merge

Para entender a implementação genérica de uma SegTree, precisamos entender alguns conceitos. O primeiro deles é o `merge`.

Sabemos que o valor de um nó depende dos valores dos seus filhos, mas isso depende de qual problema queremos resolver:

- Se estamos mexendo com o maior valor de um intervalo $[l, r] \rightarrow \text{maior_valor[nó]} = \max(\text{maior_valor[filho_esquerdo]}, \text{maior_valor[filho_direito]})$.
- Se estamos mexendo com soma de um intervalo $[l, r] \rightarrow \text{soma[nó]} = \text{soma[filho_esquerdo]} + \text{soma[filho_direito]}$.

E por aí vai. Logo, para não nos limitarmos a uma única implementação de SegTree, pensem o seguinte:

$$\text{tree[nó]} = \text{merge}(\text{tree[esq]}, \text{tree[dir]})$$

E daí, "o que é a função merge" depende do problema que você está resolvendo:

```
// Maior valor
int merge(int a, int b){
    return max(a, b);
}

// Soma de intervalo
int merge(int a, int b){
    return a + b;
}
```

```
}  
  
// ...
```

Primeira operação: Build

Primeiro, precisamos construir a SegTree a partir do vetor original.

```
vector<int> v(n);  
vector<int> tree(4 * n);  
  
void build(int node, int l, int r){  
    /*  
    "node" é o nó que queremos construir  
    "l" é o início do intervalo que aquele nó representa  
    "r" é o fim do intervalo que aquele nó representa  
    */  
  
    if(l == r){ // caso-base: estamos em uma folha  
        tree[node] = v[l];  
        return;  
    }  
  
    int m = (l + r) / 2;  
  
    // chamadas recursivas, as posições (primeiro parâmetro)  
    // seguem a regra que eu falei antes  
    build(2 * node, l, m);  
    build(2 * node + 1, m+1, r);  
  
    // calculados os valores dos filhos, atualizamos o valor  
    // atual  
    tree[node] = merge(tree[2 * node], tree[2 * node + 1]);  
}
```

Segunda operação: Query

Queremos fazer uma consulta sobre um intervalo $[l, r]$. Por convenção, vamos adotar que cada nó da árvore representa um intervalo $[start, end]$, para não confundir as letras. Ao analisar um nó, podemos cair em 3 casos.

Caso 1: O nó representa um intervalo totalmente fora do que queremos.

Suponha $[l, r] = [0, 5]$ e $[start, end] = [7, 10]$. Traduzindo, isso quer dizer que o intervalo que queremos está totalmente fora do intervalo que aquele nó representa.

A conclusão que tiramos disso, é que esse nó é inútil para a resposta. Nem o valor contido nele, nem o valor contido nos seus filhos (e netos, bisnetos...) é útil pra gente. Nesse caso, só precisamos parar a recursão e não interferir em nada no resultado final (guardem essa última frase).

Caso 2: O nó representa um intervalo totalmente dentro do que queremos.

Suponha $[l, r] = [0, 10]$ e $[start, end] = [3, 8]$. O nó que estamos representa um subintervalo da resposta que queremos. Nesse caso, continuar a recursão não faz sentido, porque estaremos olhando para subintervalos menores sendo que queremos saber do intervalo inteiro.

MAS, diferentemente do caso anterior, a resposta do subintervalo que estamos é de fato relevante para a resposta total, então retornamos o valor contido no nó atual.

Caso 3: Há uma intersecção parcial entre os intervalos.

Suponha $[l, r] = [0, 5]$ e $[start, end] = [3, 10]$. Aqui, sabemos que a resposta contida no nó atual não servirá, mas, a resposta de algum dos descendentes do nó atual pode ser que seja útil para a resposta final. Então, adentramos mais afundo na árvore.

Código

```

int query(int node, int start, int end, int l, int r){

    // Caso 1: fora do intervalo
    if (r < start || end < l)
        return NEUTRAL; // Explico o que é 'NEUTRAL' mais aba
ixo

    // Caso 2: completamente dentro
    if (l <= start && end <= r)
        return tree[node];

    // Caso 3: interseção parcial
    int mid = (start + end) / 2;

    int left = query(2*node, start, mid, l, r);
    int right = query(2*node+1, mid+1, end, l, r);

    return merge(left, right);
}

```

Elemento Neutro

Esse é mais um dos conceitos que nos ajuda a generalizar a implementação da SegTree, junto com o `merge`, o `NEUTRAL` vai variar conforme o problema que você está resolvendo. Ele precisa ser sempre um valor que não interfere em nada na sua função `merge`.

Por exemplo:

- Se estamos analisando o maior valor de um intervalo → o elemento neutro é $-\infty$. Já que $-\infty$ não interfere na função `max(a, b)`.
- Se estamos analisando a soma de um intervalo → o elemento neutro é 0. Já que 0 não interfere em `a + b`.

E assim por diante.

Terceira operação: Update

Aqui, vamos adotar a mesma convenção de intervalos, $[start, end]$ é o intervalo que um determinado nó representa.

A ideia do `update` é simples, descer na árvore até encontrar a folha $[n, n]$ em que n é igual ao índice do elemento que queremos atualizar no array, alterar o valor contido naquele nó, e depois subir na recursão atualizando os pais daquela folha com a função `merge`.

```
void update(int node, int start, int end, int idx, int value)
{
    // 'idx' é o índice do elemento que queremos alterar
    no array

    // Caso base: encontramos a folha
    if(start == end){
        v[idx] = value;
        tree[node] = value;
        return;
    }

    int mid = (start + end) / 2;

    // Decide em qual filho descer
    if(idx <= mid)
        update(2*node, start, mid, idx, value);
    else
        update(2*node+1, mid+1, end, idx, value);

    // Recalcula o nó atual após atualizar o filho
    tree[node] = merge(tree[2*node], tree[2*node+1]);
}
```